



Cyberpunk Detective

Decode the Crime

Software Requirements Specification

Version 1.0

Project Name: Cyberpunk Detective

Theme: Sci-Fi Mystery Game

Category: Unity Game Foundry

Table of Contents

1.1	Background and Necessity	3
1.2	Proposed Solution	6
1.3	Scope of Project	11
1.4	Constraints	12
1.5	Architecture Diagram	15
1.6	Flowchart	18
1.7	Functional Requirements	20
1.8	Non-Functional Requirements	31
1.9	Interface Requirements	32
1.10	Project Submission Details	33

1.1 Background and Necessity

The global gaming industry has experienced remarkable growth over the last decade, driven by rapid advancements in graphics technology, Artificial Intelligence (AI), cloud computing, and immersive storytelling. Among the fastest-growing genres, detective and mystery games have attracted players who enjoy solving crimes, investigating complex narratives, making logical decisions, and exploring highly interactive environments. At the same time, the cyberpunk genre has become increasingly popular due to its futuristic settings, advanced technologies, AI, cybernetic enhancements, and dystopian societies.



Market Trends and Analysis: Recent market research indicates that narrative-driven adventure games continue to experience steady growth due to increasing player demand for meaningful stories and interactive experiences. Modern players no longer seek only action-oriented gameplay; instead, they appreciate games that challenge their analytical thinking, decision-making abilities, and problem-solving skills.

Games featuring detective investigations, branching narratives, and realistic character interactions have demonstrated significant commercial success across PC and console platforms. The cyberpunk genre has also seen increasing popularity because of its visually stunning futuristic environments, neon-lit cities, advanced technology, and morally complex narratives.

Further, the growing popularity of AI, Machine Learning (ML), and procedural storytelling technologies allow developers to create more believable Non-Player Characters (NPCs). These technologies also support smarter investigation systems and dynamic gameplay experiences than ever.

Target Audience:

Cyberpunk Detective is designed to appeal to a diverse audience of players interested in science fiction, investigation, and narrative-driven gameplay. It is designed to attract following audience segments:



1. **Detective Game Enthusiasts:** Players who enjoy investigating crimes, collecting evidence, solving mysteries, analyzing clues, and identifying criminals through logical reasoning.
2. **Science Fiction Fans:** Players fascinated by futuristic cities, cybernetic technology, AI, robotics, holographic interfaces, and dystopian societies.
3. **Story-Driven Gamers:** Players who prefer cinematic narratives featuring emotional storytelling, memorable characters, branching dialogues, and multiple endings influenced by player decisions.
4. **Casual Adventure Players:** Gamers seeking immersive experiences that can be enjoyed in short sessions while still providing meaningful progression and engaging gameplay.
5. **Technology Enthusiasts:** Players interested in futuristic concepts such as quantum computing, neural implants, cybernetics, AI, augmented reality, and smart city infrastructures.

Technological Feasibility

Recent advancements in Unity game development technologies make **Cyberpunk Detective** highly feasible for development using modern rendering pipelines, AI systems, and advanced visual effects.

Unity's Universal Render Pipeline (URP) and High Definition Render Pipeline (HDRP) enable developers to create visual effects for a cyberpunk environment. These effects include realistic neon lighting, volumetric fog, reflective surfaces, holographic projections, and cinematic post-processing effects.

The C# programming language allows efficient implementation of investigation mechanics, inventory management, branching dialogue systems, evidence databases, mission progression, and AI behavior.

Unity NavMesh enables intelligent NPC navigation throughout complex urban environments while animation systems support realistic character interactions, facial expressions, and interrogation sequences.

Advanced shader technologies create holographic User Interfaces (UIs), digital glitches, augmented reality scanning systems, and futuristic visual effects that reinforce the cyberpunk atmosphere.

Machine Learning: Unity ML-Agents Toolkit can be used to train the security patrol agent to select from a limited set of predefined patrol behaviors based on selected player activity. The ML component is restricted to the single patrol character and does not generate dialogue, puzzles, missions, or other game content.

Player Engagement and Retention

Maintaining long-term player engagement is essential for narrative-driven detective games. **Cyberpunk Detective** incorporates several gameplay systems designed to encourage exploration, replayability, and player satisfaction. Key gameplay systems that support long-term player retention can include:

- **Branching Investigation Paths:** Players can solve investigations using different approaches. Evidence collected, suspects questioned, and choices made during missions determine how each case unfolds, encouraging multiple playthroughs.
- **Intelligent Evidence System:** Players gather simulated fingerprints, DNA samples, surveillance footage, encrypted documents, witness statements, and digital records. Each piece of evidence contributes to solving larger criminal conspiracies.
- **Interactive Crime Scenes:** Crime scenes feature hundreds of inspectable objects, environmental storytelling, hidden clues, and forensic analysis tools that reward careful observation rather than random exploration.

- **AI-Driven NPC Behavior:** Citizens, suspects, police officers, and criminals react intelligently to player actions. NPC schedules, routines, and emotional responses create believable investigations where player behavior affects the world
- **Achievements and Rewards:** Players earn achievements for solving cases without missing evidence, completing investigations within time limits, uncovering hidden conspiracies, and so on.



1.2 Proposed Solution

Nova Nexus Studios, a game development company, is developing **Cyberpunk Detective**, an offline single-player sci-fi detective adventure set in the year 2148 in Neo Zenith, a technologically advanced megacity. The game combines crime investigation, environmental exploration, light stealth elements, interactive storytelling, and logical puzzle-solving into a single narrative-driven experience.



Players assume the role of Detective Kael Orion, an investigator working for the Cyber Crime Investigation Bureau (CCIB), who is assigned a single case within one location of the city. Unlike conventional action games where combat is the primary mechanic, **Cyberpunk Detective** emphasizes intelligence, observation, and strategic decision-making. The investigation begins with an introductory objective that directs the player to the crime scene and initiates the evidence-gathering sequence. The player gathers forensic evidence, analyzes the crime scene, questions witnesses, and reconstructs events prior to identifying a suspect.

Following a focused and linear structure, the case unfolds through a single and clear resolution path built around approximately five key pieces of evidence. This keeps the experience concentrated on observation, deduction, and evidence analysis rather than branching storylines, multiple endings, or hacking mechanics.

Developed using Unity and C#, **Cyberpunk Detective** integrates cinematic lighting, realistic environmental effects, minimal AI-driven patrol behavior, and a static UI-based dialogue system. These features are intentionally scaled down from a full narrative-driven detective game to fit the available development time while still delivering a polished and immersive experience.

This project also gives Nova Nexus Studios an opportunity to build expertise in narrative-driven game development and NPC behavior. Lessons learned can be applied to future and larger scope titles.

Cyberpunk Detective contains one predefined investigation case designed by the development team. The case involves a specific crime, a defined investigation location, a limited set of suspects, approximately five evidence items, supporting characters, and a predetermined sequence of events. The development team defines the correct suspect and the evidence that supports the solution before implementation. During gameplay, the player discovers and reviews the evidence, interacts with relevant characters, solves the required puzzles, and uses the available information to identify and accuse the suspect.

It is important to note that the player cannot proceed to the suspect-identification stage until all required evidence and investigation objectives have been completed.

The game checks the player's accusation against the predefined case solution and does not dynamically generate crime, suspects, evidence, or final resolution.

Case Design Elements

Before implementation, the development team defines following elements for the single investigation case:

- **Crime:** What happened?
- **Location:** Where did it happen?
- **Victim/Target:** Who or what was affected?
- **Suspects:** Who could have committed it? A small, predefined set of suspects, including one correct suspect.
- **Evidence:** Approximately five clues
- **Puzzles:** Up to two
- **Witnesses:** Who provides information?
- **Evidence Relevance:** Each evidence item should provide a clue that supports, contradicts, or helps establish the relationship between the suspects and the crime.
- **Correct Suspect:** Fixed in the case data
- **Sequence of Events:** What actually happened?
- **Final Resolution:** What happens when the player identifies the correct suspect?

Features of Cyberpunk Detective can include:

1. **Futuristic Investigation Environment:** The game is set inside the sprawling futuristic metropolis of Neo Zenith. However, the game will be limited to one location only.

Examples include:

- Neon Commercial District
- Corporate Headquarters
- Underground Black Markets
- Residential Mega Towers
- Industrial Manufacturing Zone
- Research Laboratories
- Abandoned Subway Networks

- Cyber Slums
- AI Data Centers
- Government Security Facilities

A location can contain hidden evidence, side investigations, NPC interactions, and environmental storytelling that enrich the player's understanding of the world.

2. Puzzle-Based Gameplay: The investigation includes up to two predefined logical puzzles, such as circuit rewiring or symbol decoding. Puzzle complexity is kept simple and appropriate to the project scope.



3. Adaptive Patrol AI

The game uses a lightweight machine-learning-based AI system to control one security patrol character during the investigation.

The patrol character normally follows predefined routes but can adapt its patrol behavior based on the player's recent movement and detection history.

The AI selects from a limited set of predefined patrol actions, such as continuing the current route, investigating a recent player location, or changing to an alternate route.

The AI should operate within the single investigation environment and should not require autonomous NPC generation, complex crowd behavior, procedural decision-making, or machine-learning-based dialogue.

User Experience Features

To provide a highly engaging experience, **Cyberpunk Detective** incorporates several user-centered features designed to improve accessibility, immersion, and replay value:

1. Interactive Tutorials: The game begins with an interactive tutorial introducing:

- Movement controls
- Investigation mechanics
- Evidence collection
- Dialogue system
- Puzzle solving

Rather than interrupting gameplay, tutorials are integrated naturally into the opening mission.

2. Accessibility Features: The game includes:

- Subtitle customization
- Color-blind support
- Adjustable text size
- Customizable controls
- Aim sensitivity settings
- Heads-Up Display (HUD) scaling
- Audio balancing
- Controller support
- Keyboard remapping

These options ensure that players of different abilities can comfortably enjoy the game.

3. Immersive Audio Experience: Audio plays a crucial role in building atmosphere. The game features include:

- Dynamic ambient city sounds
- Rain effects
- Hover vehicle traffic
- Police sirens
- Electronic device sounds
- Directional footsteps

4. Achievement and Reward System: Players unlock achievements by:

- Solving investigations perfectly
- Completing cases without missing evidence
- Discovering hidden locations
- Unlocking secret dialogues
- Completing all side investigations
- Reaching multiple endings
- Finding hidden collectibles

Rewards include:

- Detective outfits
- Cybernetic upgrades
- Weapon skins
- Investigation gadgets
- New detective badges
- Digital trophies
- Character customization options

Each playthrough offers a fresh investigative experience, motivating players to explore different choices and uncover each secret hidden within Neo Zenith.

1.3 Scope of Project

The scope of **Cyberpunk Detective** is to develop a focused, offline, single-player detective game that can be completed within the available development period. The game centers on **one investigation case** taking place in **one primary cyberpunk-themed location**. The player explores the environment and interacts with relevant objects and characters. The player collects approximately **five important evidence items**, reviews the collected evidence, and completes the investigation through a predefined sequence of objectives.



The project includes basic player movement, camera control, environmental interaction, evidence collection, an evidence record, static text-based dialogue, and investigation objective management. A minimal security patrol system provides a simple stealth element through one patrol character with

predefined movement and basic player detection.

The investigation follows a **linear progression**. After completing the required evidence and investigation objectives, the player reaches a **single predetermined case resolution**. The game also provides basic save/load functionality, a pause menu, settings, and appropriate game exit options.

Note: The project does not include multiple cases, multiple locations, open-world exploration, a dedicated hacking system, branching dialogue, multiple endings, complex inventory systems, advanced AI, multiplayer, or machine-learning-based content generation. Machine learning is limited to the adaptive patrol behavior defined in the functional requirements.

1.4 Constraints

The development of **Cyberpunk Detective** involves several technical, operational, financial, and project management constraints that must be carefully considered throughout the software development life cycle. Since the game combines advanced graphics, AI, investigation mechanics, cinematic storytelling, and interactive environments, the development team must balance quality, performance, and available resources.



These constraints cover technical, operational, hardware constraints, software constraints, and save data security concerns:

1. Technical Constraints - Performance Optimization:

Technical constraints refer to the hardware, software, and technological limitations that affect the development or performance of the game. As **Cyberpunk Detective** incorporates modern rendering techniques, AI systems, detailed environments, and technical optimization plays a vital role in ensuring smooth gameplay across supported platforms.

These include:

- **Graphics Rendering and Performance:** The cyberpunk setting requires visually rich environments containing high-resolution textures, dynamic lighting, holographic effects, reflections, and weather systems. While these features greatly enhance immersion, they also increase the demand on system resources, particularly the Graphics Processing Unit (GPU) and memory.

- **AI Complexity:** AI is limited to one security patrol character. The patrol uses predefined movement and detection rules as its baseline behavior and a lightweight machine-learning model to select from a limited set of predefined patrol actions based on selected player activity. This restricted AI scope reduces computational and development complexity.
- **Environment Management:** The game uses one primary investigation environment. Limiting the environment to a single location reduces memory usage and simplifies development, testing, and optimization.

2. Operational Constraints - Development Timeline:

Operational constraints refer to the practical challenges associated with project planning, development workflow, resource allocation, and quality assurance. These factors influence how efficiently the development team can complete the project while maintaining the expected quality standards.

These include:

- **Development Timeline:** Game development consists of multiple phases including planning, programming, asset creation, testing, optimization, and deployment. Delays in any one phase can directly affect the overall project schedule.
- **Team Coordination:** **Cyberpunk Detective** requires collaboration between programmers, artists, designers, animators, audio engineers, and testers. Effective communication among team members is essential to ensure that all project components integrate successfully.
- **Quality Assurance:** Quality assurance ensures that gameplay systems function correctly across different scenarios before release. Comprehensive testing helps identify technical issues, gameplay inconsistencies, and UI problems.

3. Hardware Constraints:

Hardware limitations directly influence graphical quality, processing speed, memory usage, and overall gameplay performance. Since players use systems with varying specifications, the game must be optimized to deliver a consistent experience across supported configurations.

These include:

- **Processor Limitations:** Modern gameplay systems require significant processing power to manage AI, physics calculations, dialogue systems, and environmental interactions simultaneously.
- **Graphics Hardware:** **Cyberpunk Detective** relies heavily on advanced graphical effects to create its futuristic atmosphere. Lower-end graphics hardware struggles to render complex visual effects at acceptable frame rates.

4. Software Constraints:

The development process depends upon various third-party software applications and development tools. Compatibility between these tools is essential for maintaining an efficient workflow and preventing technical issues during development.

These include:

- **Unity Engine Compatibility:** The project is developed using the Unity Game Engine, making the development process dependent upon Unity's supported features and engine updates. Changes introduced in newer Unity versions require additional testing before adoption.
- **Third-Party Assets:** The project incorporates selected third-party assets such as sound effects, plugins, fonts, or environmental models. These resources must remain compatible with the project's technical requirements and licensing policies.

5. Security and Data Constraints:

Protecting player data and maintaining software integrity are important aspects of modern game development. Although **Cyberpunk Detective** is primarily an offline game, user information and saved progress must still be handled responsibly.

- **Save Data Security:** Player progress represents many hours of gameplay and should therefore be protected against accidental corruption or unauthorized modification. Reliable save systems improve player confidence and reduce frustration.
- **Privacy Considerations:** Even though the game operates primarily offline, any optional online functionality introduced

in future versions should comply with established privacy standards and secure communication practices.

6. Machine Learning:

The machine-learning component must be trained and tested within the available development time and hardware resources. The model should use a limited observation space and predefined actions to reduce training complexity. The game should provide a rule-based fallback behavior if the trained model is unavailable or produces an invalid result. The patrol model is trained during development using a controlled training environment and is deployed as a trained model with the game. The model is not retrained during normal gameplay.

1.5 Architecture Diagram

The architecture illustrates the overall structure of **Cyberpunk Detective** and demonstrates how the major components of the game communicate with one another during execution. It provides a high-level representation of the software architecture by showing the relationship between the player, the game engine, gameplay systems, data management, AI, and the presentation layer.

Architecture Overview: Cyberpunk Detective adopts a modular architecture in which each major gameplay feature is organized into separate layers. Each layer communicates with adjacent modules through clearly defined interfaces, ensuring that the game remains organized and easier to maintain throughout the development lifecycle.

Figure 1 shows the application architecture for the game.

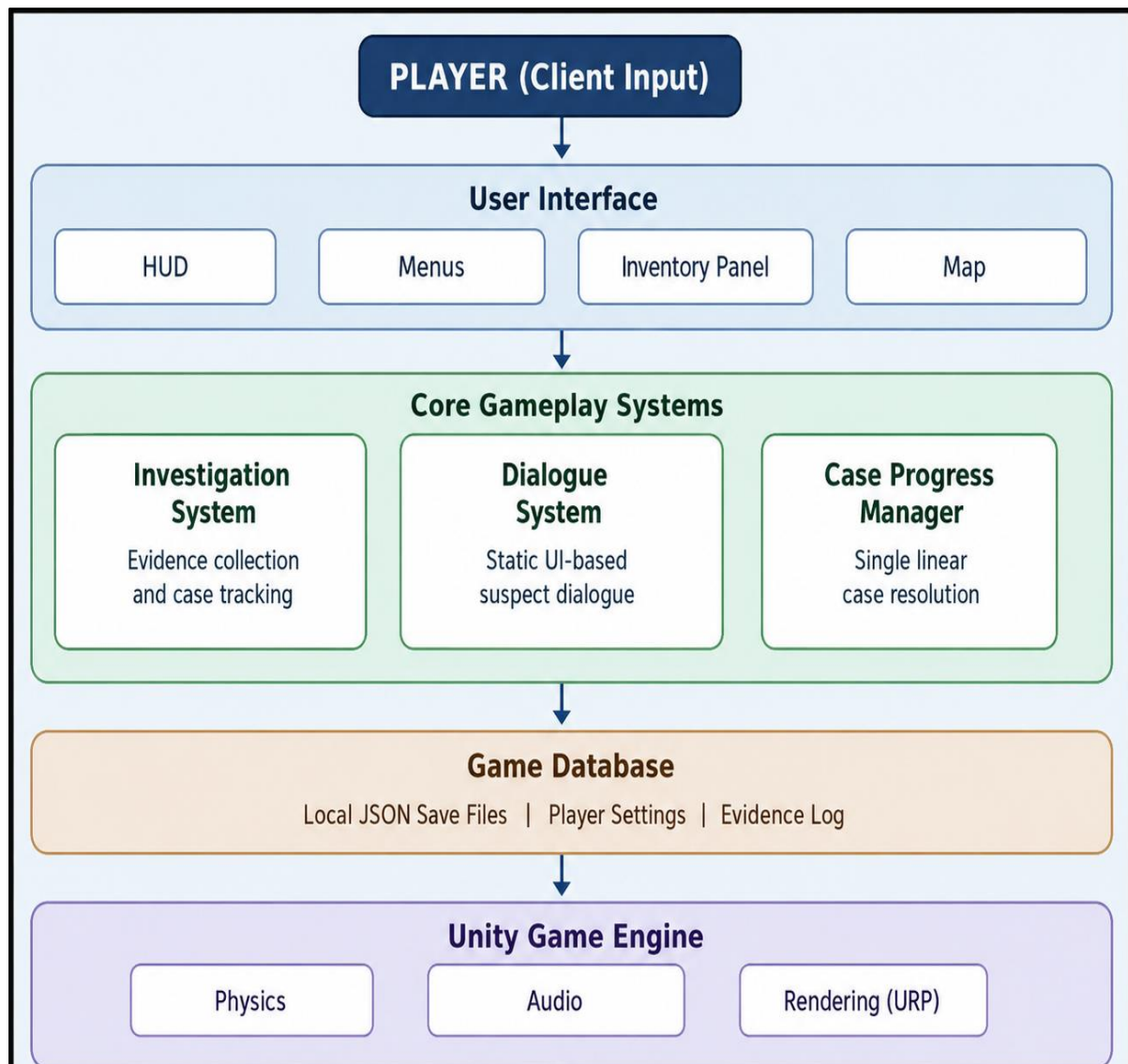


Figure 1: Application Architecture Diagram

Architecture Components: The application architecture consists of several interconnected components that work together to provide a seamless gameplay experience. Each component performs a specialized function while exchanging information with other modules whenever required. This separation of responsibilities improves software quality and simplifies future maintenance.

These components are as follows:

1. Presentation Layer: The Presentation Layer acts as the communication bridge between the player and the game. It is responsible for displaying information, receiving player input, and presenting visual feedback based on gameplay events. Each interaction initiated by the player begins within this layer before being processed by the underlying game systems.

- **UI:** The UI updates dynamically as investigations progress, ensuring that players always receive accurate information without interrupting gameplay. Clear interface design also reduces player confusion during complex investigations.
- **HUD:** The HUD is designed to minimize distractions while supporting efficient decision-making during investigations and stealth sequences. Important notifications appear contextually rather than remaining permanently on the screen.

2. Gameplay Management Layer: This layer coordinates the primary gameplay mechanics that define the player's overall experience. It acts as the central controller responsible for managing missions, investigations, interactions, and player progression throughout the game.

- Mission Management System
- Interaction System

3. AI Layer

The AI Layer controls the behavior of the security patrol character. It combines predefined patrol and detection rules with a lightweight machine-learning model that selects from a limited set of patrol behaviors based on selected player activity.

- Patrol and Detection System: Controls predefined movement, detection, and alert behavior.
- Adaptive Patrol Model: Uses selected player behavior data to choose an appropriate predefined patrol action.

3. Data Management Layer: This layer is responsible for storing and retrieving all game-related information. Reliable data management ensures that player progress, collected evidence,

mission completion, and gameplay settings remain consistent throughout the application.

- **Save Management:** Backup save functionality further improves reliability by allowing players to recover previous progress if necessary.
- **Configuration Management:** This modular structure also simplifies future updates by allowing new configuration options to be added independently.

4. Unity Game Engine Layer: The Unity Game Engine serves as the foundation of the entire application by providing rendering, animation, audio, physics simulation, input management, and scripting support. Each gameplay system ultimately relies upon Unity to execute its functionality efficiently.

- **Rendering and Animation:** Efficient rendering optimization ensures stable frame rates while maintaining visual quality across supported hardware.
- **Physics and Audio:** The physics engine controls collisions, object interactions, and environmental simulations, while the integrated audio system manages music, ambient sounds, dialogue, and sound effects. Together, these systems enhance gameplay realism and improve player immersion.

1.6 Flowchart

Figure 2 shows a broad level flowchart for the application. This flowchart shows how a player moves through **Cyberpunk Detective**, from launching the game to wrapping up an investigation and ending the session.

Gameplay Flow Overview: Each case follows a clear path - select a mission, explore and gather evidence, then analyze the case to complete it.

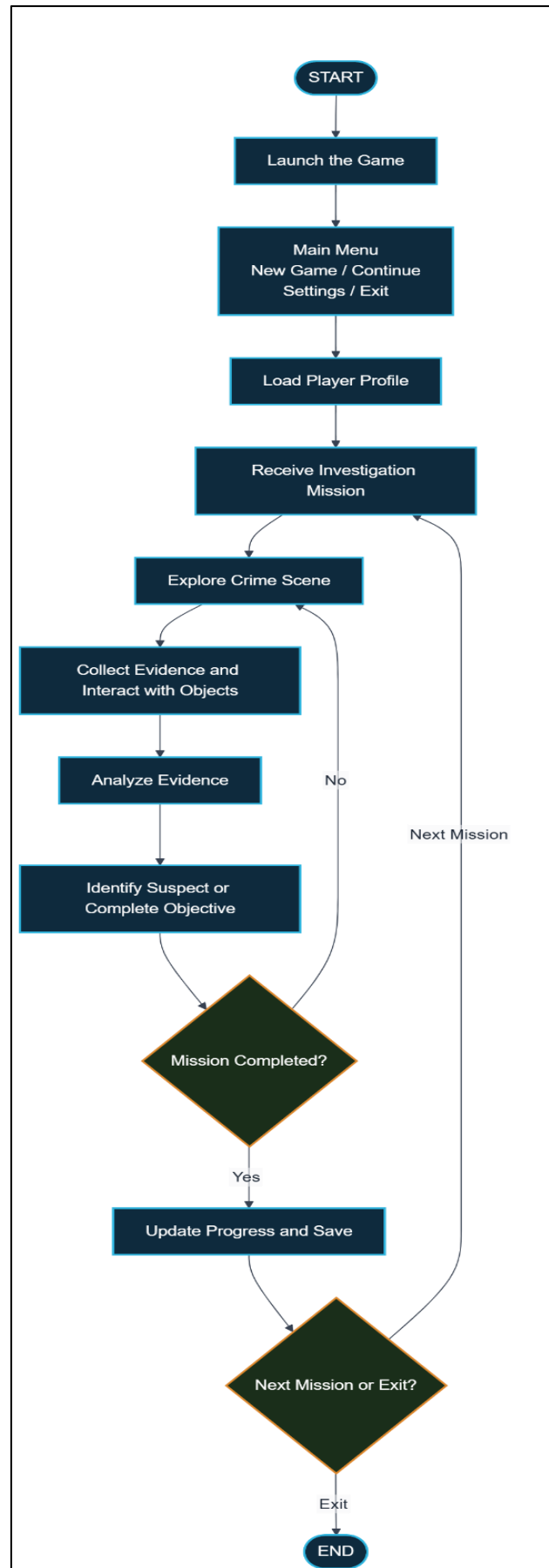


Figure 2: Flowchart for the Application

Flowchart Description: The flowchart consists of several sequential stages that guide the player from starting the application to completing investigations. Each stage performs a specific function within the gameplay loop and contributes to the overall progression of the story. Following sections explain the purpose of each stage in detail:

- **Game Launch:** The first stage begins when the player starts the application. During this process, the game loads the required resources, verifies essential configuration files, and prepares the environment for gameplay. This stage ensures that all systems are initialized correctly before the player reaches the main menu.
- **Player Session:** After entering the game, the player's profile and saved progress are loaded. This stage restores previous achievements, unlocked missions, collected evidence, and gameplay settings, so that players can continue their investigation without losing progress.
- **Mission Assignment:** The mission assignment stage also activates the required gameplay systems, ensuring that relevant characters, evidence, and interactive objects are available during the investigation.
- **Investigation Phase:** This phase combines exploration, observation, and interaction to create an engaging detective experience without forcing players into a single solution.

To summarize, the game flow will be like this:

Launch → Main Menu → New Investigation / Continue → Tutorial → Investigation → Completed Case → Case Resolution → Main Menu/Exit

1.7 Functional Requirements

Functional requirements define the specific operations and behaviors that the **Cyberpunk Detective** application must provide to the player during a complete investigation. The requirements in this section are limited to the approved project scope. These are designed to ensure that the implemented functionality can be completed, tested, and demonstrated within the development period.

Following functional requirements represent the essential capabilities that the game must implement during development:

1. Main Menu Management

The Main Menu provides the initial interface through which the player accesses the game. It should provide only the options required for starting and managing the single-player investigation.

The system should:

- Display the game title and primary menu options when the application starts
- Provide an option to start a new investigation
- Provide an option to continue an existing investigation when saved progress is available
- Provide access to basic game settings
- Provide access to project credits
- Provide an option to exit the application

The Main Menu should not contain **multiplayer**, online matchmaking, mission selection, or other features outside the approved single-case scope.

2. Interactive Tutorial

The Interactive Tutorial function introduces the basic controls and investigation mechanics during the opening section of the case.

The system should:

- Introduce player movement and camera controls.
- Display a brief instruction for interacting with objects.
- Guide the player to collect the first evidence item.
- Introduce the dialogue interaction.
- Introduce the evidence record.
- Introduce the basic puzzle interaction where applicable.
- Allow the player to continue the investigation after completing the tutorial steps.

The tutorial should be integrated into the first part of the investigation and should not require a separate tutorial level.

3. Case Initialization

The Case Initialization function establishes the single investigation when the player starts a new game.

The system should:

- Load the primary investigation location
- Place the player at the designated starting position
- Initialize the case state
- Set the initial investigation objective
- Reset previously collected evidence when a new investigation is started
- Prepare the required interactive objects and characters for the case



Only one investigation case should be active in the game.

4. Player Movement and Camera Control

The Player Movement function allows the player to navigate through the investigation location.

The system should:

- Allow the player to move within the designated investigation environment
- Provide camera control for observing the surrounding environment
- Prevent the player from moving through restricted physical areas
- Support the input method defined for the target desktop platform
- Temporarily restrict player movement when a required interaction or dialogue interface is active

The movement system should be limited to a single investigation environment and should not require open-world traversal.

5. Object Interaction

The Object Interaction function allows the player to interact with objects that are relevant to the investigation.

The system should:

- Detect when the player enters the interaction range of a designated object
- Display an appropriate interaction prompt
- Allow the player to activate the interaction
- Trigger the behavior assigned to the selected object
- Prevent interaction with objects that are not currently available for the player's progress state

This function provides the common interaction mechanism used by evidence objects and other required environmental interactions.

6. Evidence Collection

Evidence Collection is the primary investigation function of the game. The player should search the investigation location and collect the evidence required to solve the case.

The system should:

- Provide approximately five major evidence items within the investigation location
- Allow the player to inspect an evidence item before collecting it where appropriate
- Add collected evidence to the player's investigation record
- Prevent the same evidence item from being collected repeatedly
- Provide confirmation when an evidence item has been successfully collected
- Update the case progress after each required evidence item is collected

The evidence items should directly contribute to the single investigation and its final resolution.

7. Evidence Record

The Evidence Record provides the player with access to information about evidence already collected.

The system should:

- Display the evidence items collected during the current investigation
- Display a short description associated with each collected item
- Clearly distinguish collected evidence from evidence that has not yet been discovered
- Allow the player to review collected evidence during the investigation
- Maintain the evidence record until the investigation is completed or a new investigation is started

The Evidence Record should function as a simple investigation reference and not implement a complex deduction graph or relationship-management system.

8. Suspect Identification and Accusation

The Suspect Identification function allows the player to identify the suspect after completing the required investigation tasks.

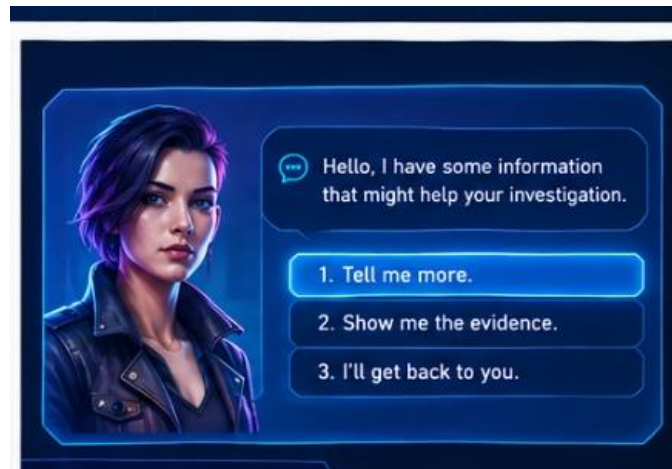
The system should:

- Display the available suspects after all required evidence and objectives are completed.
- Allow the player to select one suspect.
- Allow the player to review the relevant evidence before confirming the accusation.
- Provide an Accuse option to submit the selected suspect.
- Compare the selected suspect with the predetermined correct suspect.
- Display a success message and activate the final case resolution when the correct suspect is selected. After displaying the final case resolution, the system should provide an option to return to the main menu or exit the game.
- Display an incorrect accusation message and allow the player to review the evidence and try again.

The system should support one correct suspect and should not include multiple endings or branching case outcomes.

The correct suspect is predefined as part of the case design, while the evidence items provide clues that enable the player to

logically identify that suspect. The system checks the player's accusation against the predefined answer and does not automatically perform the deduction on behalf of the player.



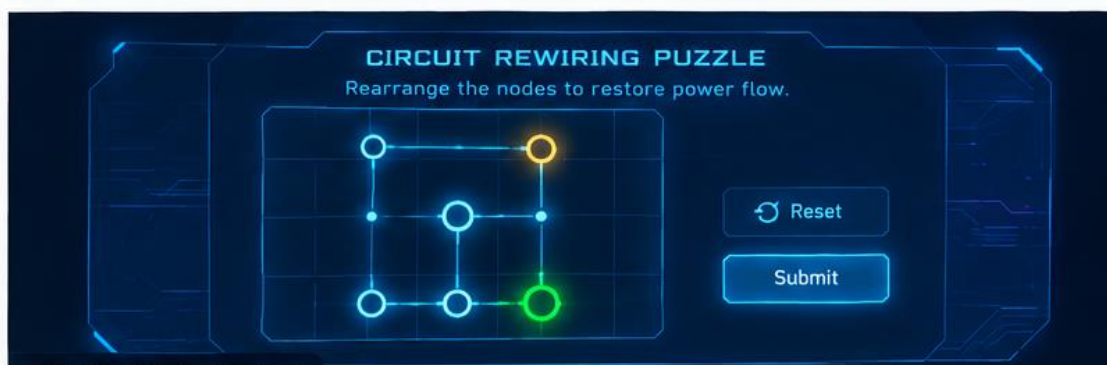
9. Static Dialogue

The Static Dialogue function provides narrative and investigative information through predefined text.

The system should:

- Allow the player to interact with designated characters
- Display predefined dialogue through a UI panel
- Present dialogue in a readable sequence
- Allow the player to advance through the available dialogue
- Close the dialogue interface when the conversation is completed

Dialogue should be predetermined and linear. The system should not require branching conversation trees, relationship tracking, voice recognition, procedural dialogue generation, or multiple dialogue outcomes.



10. Basic Puzzle Interaction

The Basic Puzzle Interaction function provides limited logical puzzle-solving as part of the investigation.

The system should:

- Provide up to two predefined puzzles during the investigation.
- Display the puzzle instructions and required interaction.
- Allow the player to submit a solution.
- Provide success feedback when the correct solution is entered.
- Allow the player to retry an incorrect solution.
- Update the investigation objective when the required puzzle is solved.

Puzzles should be predefined and independent of procedural generation. Complex puzzle-generation systems and advanced hacking mechanics are outside the project scope.



11. Basic Security Patrol and Detection

The Basic Security Patrol function provides the minimal stealth/AI component defined within the project scope.

The system should:

- Control one designated security patrol character
- Move the character along a predefined patrol route
- Detect the player when the player enters the defined detection area

- Change the character to an alert state when the player is detected
- Perform a predefined response to the detection event
- Return to the appropriate gameplay state after the detection sequence is completed
- Allow the player to continue the investigation after the predefined alert sequence ends; detection should not cause game over or reset the investigation.

The patrol should use predefined movement and detection rules as its baseline behavior.

12. Adaptive Patrol Behavior

The Adaptive Patrol Behavior function allows the security patrol to modify its behavior based on recent player activity.

The system should:

- Monitor selected player behavior relevant to patrol decisions, such as recent detection events and frequently used routes.
- Provide a limited set of predefined patrol actions.
- Use the trained machine-learning model to select an appropriate patrol action.
- Allow the patrol to adapt its behavior during the investigation based on observed player behavior. For this project, adaptation means selecting a different predefined patrol action based on selected player behavior; the system does not continuously retrain the ML model during gameplay.
- Keep patrol movement within the designated investigation environment.
- Provide a predictable fallback behavior if the ML model cannot make a valid decision.
- Ensure that ML-based behavior does not prevent the player from completing the investigation.

- 13.** The ML model should be limited to the single security patrol character and the predefined patrol behaviors. It should not generate dialogue, puzzles, missions, or other game content.

14. Investigation Objective Management

The Investigation Objective function guides the player through the required stages of the single case.

The system should:

- Display the current investigation objective
- Update the objective when a required investigation stage is completed
- Identify the evidence or interaction required for the next stage
- Prevent progression to stages that require unavailable prerequisites
- Indicate when all required investigation tasks have been completed

The objective sequence should remain linear and not create alternative mission paths.

15. Case Progression

The Case Progression function controls the overall progress of the single investigation.

The system should:

- Track the completion status of required investigation tasks
- Recognize when all approximately five required evidence items have been collected
- Enable the final investigation stage after the required conditions are satisfied
- Prevent premature completion of the case
- Maintain a consistent sequence from investigation start to final resolution

The game should contain one case and one predetermined progression path.

16. Basic Investigation Inventory

The Basic Investigation Inventory function provides a simple list of non-evidence items required during the investigation.

The system should:

- Display investigation items collected by the player.
- Add an item when the player successfully collects it.
- Display the item's name and a short description.
- Allow the player to select an item for inspection or use where required.

- Prevent duplicate collection of the same item.
- Retain collected items as part of the saved investigation state.

The inventory should continue to be a simple item list and should not include equipment management, item trading, item crafting, weight limits, item combinations, or other complex inventory features.

17. Final Case Resolution

The Final Case Resolution function completes the investigation after the required evidence and objectives have been satisfied.

The system should:

- Verify that the required investigation conditions have been completed
- Activate the final case interaction
- Display the predetermined resolution of the investigation
- Indicate that the case has been successfully completed
- Record the completion state of the investigation

The final resolution should be linear and not contain multiple endings or branching outcomes.

18. Save and Load Management

The Save and Load function allows the player to preserve and resume investigation progress.

The system should:

- Save the current investigation state when a save operation is performed
- Store the evidence collected by the player
- Store the current investigation progress
- Restore the saved investigation state when the player selects Continue
- Ensure that a new investigation starts with a fresh case state
- Only information necessary to resume the single investigation should be stored

19. Pause and Settings Management

The Pause and Settings function provides basic control over the running game.

The system should:

- Allow the player to pause the investigation
- Temporarily stop active gameplay while the pause interface is displayed
- Provide basic audio settings
- Provide available control or display settings where supported
- Allow the player to resume the investigation

The settings system should remain limited to options necessary for the final playable build.

20. Game Exit Management

The Game Exit function allows the player to safely leave the application.

The system should:

- Provide an exit option from the main menu
- Provide an appropriate exit option from the pause interface where required
- Close the application without corrupting the current game state
- Return the player to the appropriate menu or operating system environment after exiting

1.8 Non-Functional Requirements

Non-functional requirements define the quality attributes of **Cyberpunk Detective**. They specify how the game should perform rather than what functions it should provide.

Following are the non-functional requirements that must be considered while creating the game application:

1. **Seamless Performance:** The game should maintain smooth gameplay with stable frame rates and minimal loading time.
2. **Robustness and Reliability:** The game should operate without unexpected crashes and preserve player's progress correctly.
3. **Usability:** The interface should be simple, intuitive, and easy for players to navigate.
4. **Security:** The game should protect saved game data from corruption and unauthorized modification.
5. **Scalability:** The game should support future expansion, including additional missions and gameplay features.
6. **Maintainability:** The application should use modular and well-structured code to simplify updates and debugging.



These are the bare minimum expectations from the project. It is a must to implement the FUNCTIONAL and NON-FUNCTIONAL requirements given in this SRS. Once they are complete, you can use your own creativity and imagination to add more

features if required.

1.9 Interface Requirements

Hardware Requirements

Windows Machines:

- Windows 10 or Windows 11, 64-bit or higher
- Intel i5/i7 or higher; AMD Ryzen 5 or higher
- 16 GB RAM or higher
- NVIDIA GTX 1660 or higher/AMD equivalent with 4 GB Memory or higher
- 1080 p or higher resolution monitor
- 1 TB SSD
- 4 GB graphics card (DirectX 11 or higher)
- Mouse and Keyboard

MacOS:

- macOS 10.15 or higher
- Intel Core i5/i7 or Apple M1 chip
- 16 GB RAM or higher
- Radeon Pro 560 with 4 GB Memory or higher
- 1080 p or higher resolution monitor
- 1 TB SSD
- 4 GB graphics card or Apple M1 integrated GPU
- Mouse and Keyboard

Software Requirements

- **Game Engine:** Unity – version 2020.3 or higher for advanced graphics and physics simulations
- **Unity Hub:** <https://unity3d.com/get-unity/download>
- **Operating System:** Windows 10, Windows 11, macOS 10.15 or higher
- **IDE:** Visual Studio 2019 or higher
- **Graphics and Animation:** Blender (latest version) and Adobe Photoshop (latest version)
- **Audio:** Audacity (latest version) and FMOD Studio (latest version)
- **Version Control:** Git (latest version), GitHub, or GitLab
- **Data Storage:** JSON/Other

1.10 Project Submission Details

Design, build, and submit the project along with a complete project report that includes:

1. **Problem Definition:** Clearly state the problem addressed by **Cyberpunk Detective**.
2. **Design Specifications:** Detailed design documents and diagrams (such as flowcharts and data flow diagrams).
3. **Installation Instructions:** Step-by-step guide for installing **Cyberpunk Detective**.
4. **User Credentials:** Share the test user accounts for review.

Project Files: Include project files (source code), Unity packages, and any additional resources required. Submit this as a consolidated zip file.



Ensure that all documentation is thorough and clear, providing a comprehensive overview of the game's functionality, architecture, and usage.

Documentation should not contain source code.

Submit a video (.mp4 file) demonstrating the working of the game application, including all the functionalities of the project. This is MANDATORY.

~~ End of the Document~~